TAXOSHOP: Mining Recurring Failure Patterns for Cross-Interpreter Bug Discovery

***Abstract*—Interpreted languages such as Python, Ruby, and PHP are fundamental to modern software development, from powering servers to machine learning applications. CPython's repository in particular contains years of bug reports and related patches; however, traditional bug variant analysis techniques examine bugs individually, limiting their ability to generalize vulnerable patterns. We present TAXOSHOP, an approach that uses large language models to mine a project's bug history into a taxonomy of recurring bug patterns. TAXOSHOP abstracts bug reports into implementation-agnostic workflows that are clustered by similarity. For each cluster, TAXOSHOP generates a taxonomy entry that describes the shared bug patterns as a causal chain. Applied to over 300 CPython bug reports, TAXOSHOP clusters reports and identifies systemic fault patterns. To demonstrate the taxonomy's utility, we use it to guide a simple LLM-based variant analyzer. The analyzer discovers 23 previously unknown bugs: 10 in CPython, 8 in CRuby, and 5 in the official PHP implementation, all confirmed by maintainers. Although mined solely from CPython, the taxonomy transfers across interpreters, and enables superior performance to unguided, CWE-guided, and instance-guided baselines.**

I. INTRODUCTION

Interpreted languages like Python, Ruby, and PHP are essential to modern software. They power web backends, data science pipelines, machine learning infrastructure, and a large share of the world's websites [1]. Python in particular has become one of the most widely used programming languages [2], and its reference implementation, CPython, is used by millions of applications and packages.

Their widespread usage makes the correctness and security of their implementations important. The dominant implementations of these languages (CPython, CRuby, and the Zend Engine behind PHP) are large codebases that expose rich object models to user code, and interleave interpreter-internal state with arbitrary user-defined behavior. While these interpreters provide memory safety and sandboxing measures, bugs within their implementations can undermine their security. Particularly, memory-safety bugs have historically produced denial-of-service, arbitrary code execution, and information disclosure in production environments [3]. Thus, defects in the interpreter affect every downstream application.

Simultaneously, these projects have accumulated a rich history of failures. CPython's GitHub repository alone has approximately 27,000 bug reports spanning more than 25 years. Each report describing a triggering input, the faulty behavior, and often a root-cause analysis with fix. Past bug reports encode recurrent bug patterns, and those patterns can guide the discovery of new bugs.

Variant analysis seeks to leverage this insight. Traditional approaches rely on security experts to inspect a vulnerability, identify its characteristics, and manually audit code or construct analysis rules [4]. Automated systems reduce this burden by deriving search signatures, similarity representations, or static-analysis queries from a vulnerable program and its patch [5], [6], [7], [8], [9], [10].

Techniques are largely instance-guided (e.g., clone-based detectors [11], [7], patch-signature methods [12], and directed fuzzers [13]), seeding their search with a single bug. Because such techniques primarily extrapolate from *individual* bugs, their analysis is constrained to the vicinity of a known instance. Yet, two bugs can arise from the same underlying programming mistake while sharing little source-level structure, occurring in unrelated subsystems or involving different APIs. Thus, searching for variants of individual bug instances can miss bugs that share its failure mechanism but not its surface form. Even approaches that explicitly derive higher-level variant patterns construct those patterns from individual vulnerabilities rather than discovering high-level, recurring mechanisms across a historical bug corpus [4].

Idea. A project's bug history contains patterns of recurrent flaws. If the mechanism behind each bug could be captured in a form that is independent of the specific function, type, or file in which it appeared, then the recurring mechanisms across the whole history would describe where the project, and others like it, is likely to fail again. Such a taxonomy of mechanism descriptions could guide variant analysis toward families of latent bugs at once. Furthermore, because it abstracts away implementation specifics, it could apply even to a different interpreter than the one it was mined from.

Challenges. Realizing this idea requires solving three problems. **(1) Mechanism-level abstraction:** a bug report describes a local failure in project-specific terms. Abstracting it to a comparable mechanism means discarding incidental detail while preserving the causal structure. Too much abstraction yields generic weakness classes (e.g., "use-after-free") that cannot direct a search while too little yields a description that matches only the original bug. **(2) Grouping without a known target:** the set of recurring mechanisms is not known in advance, so the bugs must be grouped by a pattern that is itself being inferred. Surface similarity can be misleading as reports of one mechanism often share no vocabulary, while reports touching the same module may be unrelated. **(3) Actionable guidance:** to guide a search, each pattern must state both failure and context, while remaining faithful to the evidence.

A description that overgeneralizes beyond what its member bugs support sends the search after bugs that do not exist.

Approach. We present TAXOSHOP, which addresses these challenges. (1) TAXOSHOP abstracts each normalized bug report into a *bug workflow*, a causally ordered description of the failure without project-specific identifiers. (2) It clusters these workflows with a hybrid method that first groups them by embedding similarity and then applies an agent to merge and split clusters by shared mechanism. (3) For each cluster, it generates a taxonomy entry by reading the full evidence of every member bug and expressing the shared mechanism. The result is a taxonomy of recurring, mechanism-level bug patterns, mined automatically from history and directly usable for variant analysis.

Results. TAXOSHOP generates a taxonomy from 363 mined CPython bug reports. Pairing this with an agentic variant analyzer, TAXOSHOP-VA, we discover 23 confirmed and previously unknown bugs across three interpreters: 10 in CPython, 8 in Ruby, and 5 in PHP. Compared with baseline agentic analysis strategies (unguided, instance-guided and CWE-guided), TAXOSHOP-VA makes nearly 3× as many discoveries as the next closest system. An ablation study further shows that our hybrid clustering technique is central to TAXOSHOP's effective bug grouping.

In summary, this paper makes the following contributions:

- **A mining approach** that uses LLMs to abstract interpreter bug reports into high-level bug workflow descriptions and clusters them into bug taxonomies.
- **An empirical taxonomy** of CPython memory-safety bugs, derived from 363 bug reports, along with an analysis of their characteristics.
- **A taxonomy-guided agentic system** that uses these categories to direct the search for new bugs in CPython and other C-based interpreters.
- **An evaluation** demonstrating the discovery of 23 new bugs across CPython, Ruby, and PHP, showing that bug patterns transfer across interpreter implementations.
- **An open-source artifact** containing our mined dataset, taxonomy, and tooling.

II. BACKGROUND

Interpreted Languages. The reference implementations of Python, Ruby, and PHP (CPython, CRuby, and the Zend Engine) are all large C codebases. Each interpreter implements a dynamic object model in native code. Built-in types, core modules, and extensions that wrap external libraries written in C manipulate interpreter objects directly. Memory-safety defects in these implementations have historically enabled attacks ranging from denial of service to arbitrary code execution (e.g., use-after-free bugs in CPython's decoders [16], buffer overflows in CRuby's `pack.c` [17], and memory corruption in the Zend Engine's allocator [18]). These interpreters share two properties that make memory-safety bugs common and structurally similar.

First, native code regularly calls back into user code. Operations that appear primitive at the language level, such as comparing two values or iterating over a container, are dispatched to methods that users can override. For example, Python's `__eq__`, `__hash__`, and `__index__` [19]; Ruby's implicit conversion methods `to_ary`, `to_int`, and `to_str` [20]; and PHP's magic methods (e.g., `__toString`, `__destruct`) and user-supplied callbacks [21]. A native routine that invokes such a method hands control to arbitrary code. That code can manipulate any object the routine is still using.

Second, all three interpreters present a garbage-collected surface where their native code is responsible for validating objects and buffers. CPython and PHP use reference counting supplemented by a cycle collector where native code must track which references it owns. CPython makes this explicit with its distinction between *owned* and *borrowed* references. A borrowed reference is valid only as long as some other owner keeps the object alive. CRuby uses a generational, incremental mark-and-sweep collector that scans the native stack conservatively. Native code must keep objects reachable and must not retain interior pointers into an object's storage across calls that can reallocate it. In each case, a borrowed reference, raw pointer, or cached length that outlives the state it refers to becomes a dangling handle.

Concurrency adds further complexity. CPython and CRuby serialize most execution under a global lock, but native code can release the lock around blocking operations. Both runtimes also support execution contexts outside a single global lock. CPython's subinterpreters can each hold their own lock [22], and CRuby's Ractors execute in parallel. CPython additionally offers an officially supported free-threaded build without a global lock [23], [24], which exposes shared state that the lock previously protected implicitly.

Variant Analysis. Variant analysis is the practice of finding new bugs that share a root cause with a known one. Bugs often do not occur in isolation. Rather, the flawed logic behind one bug frequently recurs in other locations within the same codebase. Traditionally, variant analysis is performed manually. An expert studies a fixed bug, generalizes its root

Paper	Year	Automated	Corpus-derived	Cross-project	LLM-based
ReDeBug [11]	2012	●	·	◗	·
AFLGo [13]	2017	●	·	·	·
VUDDY [7]	2017	●	·	◗	·
MVP [12]	2020	●	·	◗	·
Big Sleep [14]	2024	●	·	·	●
IRIS [15]	2025	●	·	·	●
GRAPE [4]	2026	◗	◗	●	·
TAXOSHOP	2026	●	●	●	●

TABLE I: TAXOSHOP compared with prior variant analysis approaches. ● = yes, ◗ = partial, · = no. *Corpus-derived*: guidance is abstracted from many historical bugs rather than a single seed. *Cross-project*: the derived knowledge locates bugs in projects other than the one it was mined from.

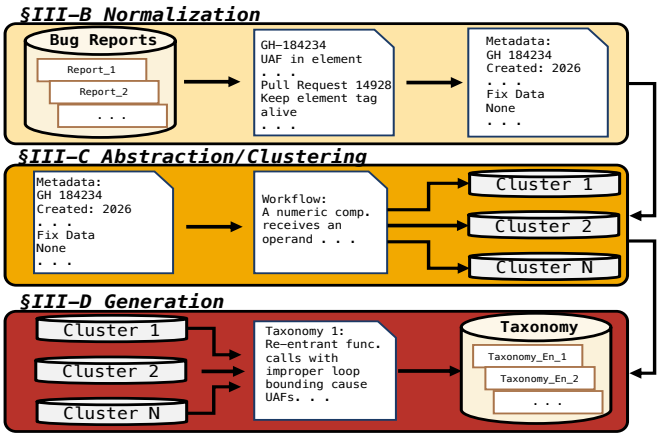


Fig. 1: Overview of TAXOSHOP

cause, and either audits related code or writes a query that encodes the pattern in a static-analysis framework such as CodeQL [25] or Semgrep [26]. The quality of the result depends on the expert's generalization, and each query is typically written for a single bug.

Automated approaches generally match code against individual known vulnerabilities. Vulnerable code clone detectors such as ReDeBug [11] and VUDDY [7] locate code that is syntactically similar to a known vulnerable function. Patch-based techniques such as MVP [12] derive signatures from a vulnerability and its fix to find unpatched instances. Directed fuzzers such as AFLGo [13] concentrate testing effort near locations associated with known bugs. More recently, LLM-based agents have been applied to variant analysis, starting from a single recently fixed bug and searching for similar code [14]. GRAPE [4] abstracts a report into a structured pattern for matching, but writes each pattern from a single seed. Table I summarizes current techniques and their features.

These approaches share two limitations. First, they operate at the granularity of a single bug. Thus, they find variants that resemble that bug's surface form but miss bugs that share its root cause through different code. Second, their signatures, queries, and seeds are tied to one codebase's identifiers and idioms, so knowledge does not transfer across projects. TAXOSHOP addresses both limitations by abstracting over many bugs at once and away from the project in which they were found.

III. TAXOSHOP

This section presents the detailed design of TAXOSHOP. We first summarize the three core challenges in generating a taxonomy for variant analysis then describe the three stages of TAXOSHOP along with its paired variant analyzer. Figure 1 illustrates the overview of TAXOSHOP.

A. Challenges

Automatically generating a practical taxonomy which can guide variant analysis introduces three primary challenges, each of which motivates the design of TAXOSHOP.

```
## Metadata
CPython issue #127971: heap-buffer-overflow in
ucs2lib_default_find; reproduced on Python 3.12 and 3.13
on Linux and macOS.
## Discussion
A Unicode str.replace() operation triggers an
AddressSanitizer error. After a candidate match fails at
the buffer boundary, the search heuristic reads the
following character without first checking that it lies
within the buffer.
## References
Issue #127971 contains the reproducer and sanitizer trace;
PR #132574 provides the fix and regression test.
## Fix
Add a boundary check before lookahead accesses in
Objects/stringlib/fastsearch.h, and add a regression test
for replacement at the buffer boundary.
```

```
## Steps
1. A text-search operation scans candidate starting
   positions within a bounded character buffer.
2. A candidate match fails at the final valid position,
   but the skip heuristic examines the following
   character to choose the next advance.
3. The lookahead occurs without checking whether the
   following character remains inside the buffer.
4. The one-past-end read causes a heap-buffer-overflow
   and may terminate instrumented execution.
## Summary
A string search reads one character past the end of its
buffer when a skip heuristic looks ahead after a failed
match at the final valid position.
```

Fig. 2: A normalized CPython report excerpt (top) and the workflow signature TAXOSHOP abstracts from it (bottom).

C1: Abstraction Level. The major inhibitor of current variant analysis techniques is fine-grained bug extrapolation. Because these techniques emphasize instance-based variants, they uncover bugs near the original instance but neglect the discovery of the same bug pattern in a different subsystem or adjacent application. Conversely, general-purpose weakness taxonomies such as the Common Weakness Enumeration (CWE) overgeneralize. For example, simply characterizing a bug as a use-after-free (CWE-416) does not provide actionable information on how to discover similar errors.

C2: Pattern Inference. To create an accurate taxonomy, recurrent bug patterns must be inferred. Determining underlying patterns requires clustering similar bugs before examining clusters from a macro-perspective. However, reports of the same mechanism rarely share vocabulary, so surface-level similarity groups bugs by module instead of root cause. Furthermore, a single report cannot reveal which of its details are essential to the pattern and which are incidental; this can only be determined by comparing every report in a cluster.

C3: Taxonomy Structure. A taxonomy must formally characterize bug clusters. Characterizing a cluster by its symptoms is insufficient, as the same root cause can manifest as different symptoms (e.g., a stale reference may cause either a use-after-free or an out-of-bounds read). Each entry must instead capture the causal chain leading to the failure, and it must be actionable.

B. Report Normalization

Practically, repository bug reports are heterogeneous in structure and level of detail. TAXOSHOP therefore normalizes each report into a uniform, agent-readable document. This stage is deterministic and purely structural. TAXOSHOP collects the information associated with a report and renders it into a fixed layout without interpreting the underlying bug. Critical evidence, including sanitizer traces, reproducer code, and fix patches, is preserved verbatim. Deferring all abstraction to later stages allows agents to reason from the original artifacts. For each report, this stage collects:

- **Metadata:** issue metadata, including the title, labels, status, and creation date.
- **Discussion:** the complete issue body, discussion comments, and linked attachments such as logs and reproducer scripts.
- **References:** referenced pull requests, commits, and other fix links.
- **Fix:** details of each referenced fix, including its description or commit message and the patch for every changed file.

The top portion of Figure 2 illustrates an excerpt of a normalized report.

C. Abstraction and Clustering

Normalized reports are detail-rich but unsuitable for comparison. Two reports with the same underlying failure mechanism in different modules share little vocabulary. TAXOSHOP therefore abstracts each report into a *workflow signature*, a short, implementation-neutral description of the bug. These workflow signatures are subsequently clustered for later stage analysis.

Workflow Abstraction. For each normalized report, an LLM produces an abstract signature consisting of an ordered sequence of steps and summary. These abstractions describe the causal sequence that leads to the failure and remove project-specific terminology, including function, API, and test names.

Formally, given normalized reports $\mathcal{R} = \{r_1, \ldots, r_n\}$ from a project P, TAXOSHOP maps each report to a workflow signature $s_i = \alpha(r_i) = (\omega_i, \sigma_i)$, where $\omega_i = \langle e_{i,1}, \ldots, e_{i,k} \rangle$ is a causally ordered sequence of $2 \leq k \leq 8$ steps ending in the failure, and σ_i is a one-sentence summary. Signatures are instance-neutral. For the set $\mathcal{V}_P$ of identifiers in P (e.g., function, API, and test names), $\mathrm{id}(s_i) \cap \mathcal{V}_P = \emptyset$.

The bottom of Figure 2 illustrates a workflow generated from a normalized report, abstracting away the report specifics into a causal chain of steps.

Hybrid Clustering. TAXOSHOP clusters signatures in two phases. The first phase is embedding-based: TAXOSHOP embeds each signature and groups signatures by cosine similarity, producing an initial partition. The second phase is agentic refinement. An LLM examines the initial clusters and reassigns or consolidates them, merging clusters whose members share an underlying failure mechanism and separating members grouped only by surface similarity. The model is instructed to treat two reports as equivalent only when their failure workflow is the same, even if their symptoms, affected files, or implementation details differ.

The two phases are complementary. Embedding similarity scales to the full corpus but can over-fragment, yielding low-population or singleton clusters. Signatures that share a failure mechanism can still fall below a single global similarity threshold. The result is an over-fragmented partition with many low-population or singleton clusters. The refinement phase corrects this. The model compares clusters by failure mechanism specifically, judging which details are essential to the pattern and which are incidental, and consolidates fragments that share a mechanism. Deferring the final grouping to the model also avoids fixing a similarity threshold in advance as the definitive boundary.

D. Taxonomy Generation

To characterize the clusters, the final stage turns each cluster into a *taxonomy entry* that describes the cluster's recurrent bug pattern. The collection of entries forms the taxonomy. Entries serve two practical purposes. For researchers, an entry characterizes a recurring way in which the project fails. For auditors, it describes how to search for new instances.

Formally, clustering partitions $\mathcal{R}$ into clusters $\mathcal{C} = \{C_1, \ldots, C_m\}$. The taxonomy is the set of all entries, $\mathcal{T} = \{t_1, \ldots, t_m\}$. For each cluster, an LLM reads the full normalized document of every member report. From this information, the model determines which features are shared across the cluster and which are specific to individual reports, and produces an entry

$$t_j = \gamma(C_j) = (\pi_j, \chi_j),$$

where π_j names and briefly describes the pattern and $\chi_j = (pre, op, int, asm, fail, scope)$ decomposes it into a contextual set:

- **Precondition** (pre): the state the code must be in for the bug to be possible.
- **Critical operation** (op): the action during which the bug is triggered.
- **Interference** (int): the event that invalidates the code's expectations during that operation.
- **Invalid assumption** (asm): the implicit belief that the interference violates.
- **Failure** ($fail$): the resulting misbehavior.
- **Scope** ($scope$): the boundary of the pattern, stating how far the shared mechanism generalizes across the member reports and which cases fall outside it.

The first five fields serve to separate the bug's conditions from its individual instances and effects while the scope bounds the generalization. The scope makes the entry actionable, telling an auditor where a candidate stops being an instance of the pattern.

E. Variant Analyzer

To demonstrate the practical utility of TAXOSHOP's taxonomy generation, we pair TAXOSHOP with a simple variant

analyzer using a flag-verification loop [27]. We refer to the analyzer guided by TAXOSHOP taxonomies as TAXOSHOP-VA.

Location Flagging. The agent selects a taxonomy entry and uses it to audit the interpreter source for code matching the pattern. For example, for the stale-state entries, it looks for native routines that retain a pointer, length, or borrowed reference across a call that can execute user code. Each candidate is recorded as a flag consisting of a file, a line range, and the taxonomy entry it instantiates. The agent keeps a running log of flagged locations and of modules it has audited and judged safe, so that later iterations do not re-audit the same code.

Flag Validation. For each flag, the agent writes a reproducer in the target language that attempts to trigger the suspected bug and runs it on a sanitizer-instrumented build of the interpreter. A flag is confirmed only if its reproducer triggers a sanitizer report whose fault location matches the flagged code. For each confirmed flag, the agent searches the project's issue tracker and commit history for existing reports of the same defect and discards duplicates. It then writes a report containing the root cause, the minimal reproducer, the full sanitizer output, and the taxonomy entry it instantiates. We review each report manually before disclosure.

IV. EVALUATION

We implemented TAXOSHOP with approximately 1K lines of Python code. TAXOSHOP uses GPT-5.6 Luna for the workflow description generation, clustering, and taxonomy generation. Claude Opus 4.8 is used for the agentic bug finding phase. We use `all-MiniLM-L6-v2` as the embeddings model. The input concatenates the sentence summary and numbered workflow steps, and the resulting embeddings are normalized to unit length. TAXOSHOP forms a graph with an edge between workflows whose cosine similarity is at least 0.72, then treats each connected component as a cluster. As a bug oracle, generated programs are executed on an ASan/UBSan-instrumented build of each interpreter. We record sanitizer reports for triage and upstream reporting.

In this section, we evaluate TAXOSHOP to assess its ability to taxonomize and uncover new bugs in C-based interpreters and to understand the contribution of each design choice. Our evaluation answers the following research questions:

- **RQ1:** Can TAXOSHOP generate a taxonomy from a corpus of bug reports?
- **RQ2:** Can a taxonomy generated from an interpreter enable discovery of new bugs within that system? Can a taxonomy enable cross-system bug discovery?
- **RQ3:** How much does the taxonomy contribute compared with baselines?
- **RQ4:** What is the effect of the clustering strategy?

A. Experimental Setup

TAXOSHOP/TAXOSHOP-VA is deployed on a system with 8 cores and 32 GB of RAM running Ubuntu 22.04. Analyses are performed on the latest development versions of CPython, CRuby, and the official PHP interpreter to ensure all findings reflect current behaviors.

Answering RQ1. To assess if TAXOSHOP can generate a taxonomy from a set of bug reports, we collect all memory-safety issues and their associated fixes from the CPython repository and use them as input to TAXOSHOP. At the time of collection, 363 CPython issues were related to memory-safety. We report the resulting cluster distribution and characterize the largest categories.

Answering RQ2. To practically evaluate the validity of the generated taxonomy, we test TAXOSHOP-VA to see if the taxonomy can guide new bug discovery in CPython. Furthermore, to assess the generalizability of the taxonomy, we apply TAXOSHOP-VA to CRuby and PHP without running TAXOSHOP on either's bug history. TAXOSHOP-VA receives a $20 API token budget per interpreter. To ensure findings reflect current interpreter behaviors, we run TAXOSHOP-VA on the daily-updated development versions of each interpreter.

Answering RQ3. To isolate the contribution of the taxonomy, we compare TAXOSHOP-VA against the same variant analyzer given alternative guidance, under the same $20 token budget on CPython[1]:

1) **Unguided:** a generic instruction to find memory-safety bugs.
2) **CWE-Guided:** descriptions of the CWE classes observed in the corpus.
3) **Instance-Guided:** all 363 individual CPython bug reports.

Answering RQ4. Clustering is crucial for grouping bugs into a usable taxonomy. To examine the effects of our hybrid clustering strategy, we compare against (1) embeddings-only clustering and (2) LLM-as-judge clustering.

Bug Oracle. For memory and logic bug detection, we instrument each interpreter with `clang`'s AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan), both of which provide precise fault localization.

Responsible Disclosure. All bugs are responsibly disclosed to the appropriate development team following their respective guidelines. Each reported bug is unique as confirmed by development teams, requiring individualized patches.

B. Results

1) CPython Taxonomy: To assess TAXOSHOP's ability to generate a taxonomy from bug reports, we mine 363 memory-safety bug reports and their associated fixes from CPython's GitHub repository and process them with TAXOSHOP.

Distribution. TAXOSHOP groups the 363 reports into 116 clusters. 52 clusters (45%) are singletons. 175 reports (48%) belong to clusters with five or more members. TAXOSHOP therefore finds many memory bugs in CPython to be idiosyncratic, each arising from a specific wrong assumption in a specific location. A number of mechanisms, however, recur

[1] commit 7bfa97f4dca58ab45def2ca0a1e8088997a4fa0b

Rank	Cluster	Count	Description
1	C-0014	38	Stale borrowed handles after invalidation. Code retains a non-owning pointer, reference, iterator, view, or cache entry while another operation can free, replace, resize, or tear down its target. Later use of that handle produces use-after-free, invalid reads, or corruption.
2	C-0080	22	Callback re-entrancy invalidates active native state. Native code calls user-defined behavior while holding borrowed references, raw pointers, cached sizes, iterator state, or container metadata. The callback mutates or destroys that state, and the native operation resumes with stale assumptions.
3	C-0137	20	Re-entrant user-code execution invalidates retained native state. A native operation crosses a user-code boundary for conversion, iteration, comparison, or callback dispatch while retaining a pointer, cached size, container slot, or resource handle. Re-entrant code mutates, detaches, closes, replaces, or releases that state before subsequent native use.
4	C-0077	11	Concurrent contexts race on process-wide state. Threads or isolated runtime contexts share lifecycle state, caches, registries, flags, metadata, or resources without complete synchronization. Initialization, publication, operation, or teardown overlaps, producing data races and potentially inconsistent state, hangs, or invalid accesses.
5	C-0091	10	Error paths lose ownership of temporary state. A native operation allocates or acquires an intermediate value, then encounters invalid input, a failed callback, or another exceptional condition. Incomplete cleanup leaves the allocation or reference live.
6	C-0079	9	Ownership-bound cleanup fails to reclaim direct or nested allocations. Destructors, local cleanup, or backend-specific release paths do not fully discharge ownership. Mechanisms include incorrect destructor metadata, shallow destruction, omitted release or transfer, and unreclaimed nested allocations, producing leaks.
7	C-0017	8	Unchecked representation assumptions across dynamic boundaries. Low-level code treats flexible or dynamically obtained values as a specific internal representation without validating their type or layout. Related cases replace type metadata on existing storage without establishing layout compatibility, allowing later operations to interpret incompatible state.
8	C-0064	8	Lifecycle ownership diverges across isolated contexts. Reusable runtimes mix context-owned resources with process-global, legacy, or externally shared state. Teardown either leaves auxiliary allocations unreclaimed or invalidates state still retained elsewhere, producing leaks or stale-state memory faults.
9	C-0100	8	Concurrent mutation or release of shared state. An operation accesses a shared descriptor, pointer, dispatch value, buffer, object field, or external-library state while another path can change or invalidate it. Inadequate synchronization or lifetime protection causes data races and, in some cases, stale or freed-state access.
10	C-0241	7	Concurrent unsynchronized access to mutable shared storage. Multiple threads retain aliases to shared buffers, containers, iterators, fields, pointers, or external-library objects. At least one operation mutates the state while another accesses it without a complete common synchronization protocol, causing races and potentially inconsistent state or invalid accesses.

TABLE II: Top ten CPython bug clusters identified by TAXOSHOP.

systemically. The five largest clusters (`C-0014`, `C-0080`, `C-0137`, `C-0077`, and `C-0091`) together account for 101 reports, or 28% of the corpus.

Dominant Patterns. Table II lists the ten largest clusters with their populations and summaries. For each cluster, TAXOSHOP produces a taxonomy entry that describes the pattern as a causal chain (precondition, critical operation, interference, invalid assumption, and failure) and includes a scope note that bounds the generalization. The primary recurrent patterns identified by TAXOSHOP are:

1) **C-0014: Stale borrowed handles after invalidation.** Code retains a non-owning pointer, reference, iterator, view, or cache entry without ensuring that its target remains valid. An intervening operation frees, replaces, or resizes the underlying object. The invalidation can arise through mutation, concurrent execution, or lifecycle transitions. Subsequent code uses the retained handle as though its ownership and lifetime assumptions still hold. The resulting failures include use-after-free, out-of-bounds access, and memory corruption.
2) **C-0080: Callback re-entrancy invalidates active native state.** A native operation invokes user-defined behavior while retaining borrowed references, raw pointers, cached sizes, or iterator state. The callback re-enters the runtime and mutates the state that the outer operation still requires. When execution resumes, the outer operation continues with assumptions established prior to the callback. The invalidated state may concern object lifetime, storage layout, container size, or traversal position.
3) **C-0137: Re-entrant conversion or protocol execution invalidates retained state.** A native operation crosses a user-code boundary while holding a pointer, cached bound, container slot, or backing-resource handle. Such boundaries include argument conversion, iteration, comparison, and other protocol operations that can execute arbitrary code. Re-entrant execution mutates, detaches, or replaces the state before the native operation finishes. The operation then resumes using its earlier view without establishing that the resource remains valid.
4) **C-0077: Concurrent contexts race on process-wide state.** Threads or isolated execution contexts share runtime metadata, caches, registries, flags, or native resources. Their apparent isolation does not provide synchronization for this process-wide state. Concurrent initialization, publication, mutation, or teardown can therefore overlap with another context's access. The failures range from sanitizer-detected races to inconsistent state, hangs, or invalid resource accesses.
5) **C-0091: Error paths lose ownership of temporary state.** A native operation acquires an allocation, reference, or other temporary resource before completing fallible work. Invalid input, failed validation, or a callback exception then diverts execution into an error path. That path fails to release the resource or transfer its ownership correctly. Repeated failures can accumulate leaked allocations or references even when successful execution cleans up correctly.

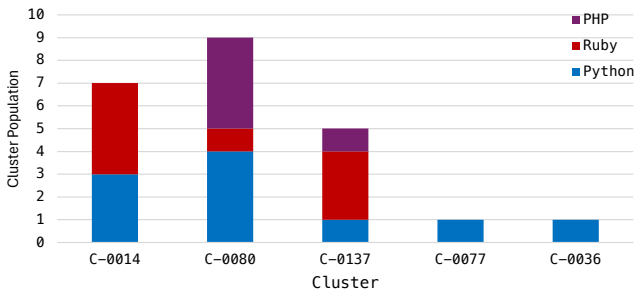


Fig. 3: Cluster Distribution of New Bugs

Answer to RQ1: Despite a group of singleton clusters, TAXOSHOP is able to identify and characterize a number of independent bug patterns given a corpus of reports.

2) Bug Discovery: To assess the practical utility of TAXOSHOP's generated taxonomy for variant analysis, we deploy TAXOSHOP-VA on CPython. To further assess the generalizability of the taxonomy, we run TAXOSHOP-VA on CRuby and PHP. Table III displays the previously unknown bugs discovered by our system.

CPython Bug Discovery. TAXOSHOP-VA discovers 10 previously unknown bugs in CPython. The discovered bugs are spread across various modules and span five weakness classes. Use-after-free (CWE-416) dominates with six bugs. The remaining four are an out-of-bounds read (CWE-125), an out-of-bounds write (CWE-787), a data race (CWE-362), and a null pointer dereference (CWE-476). At the time of writing, maintainers have confirmed all ten bugs and patched nine. With a $20 budget, this amounts to an average cost of $2 per confirmed bug.

Pattern Coverage. Every discovered bug is an instance of a TAXOSHOP taxonomy entry with TAXOSHOP-VA being guided by that specific entry during the search. Three bugs belong to `C-0014`, four to `C-0080`, one to `C-0137`, one to `C-0077`, and one to `C-0036`. Three examples illustrate how the entry directed the search. `C-0014` guided the discovery of Bugs 3 and 4: while a buffered read is in progress, the underlying raw stream's user-defined read method calls `detach()` on the buffered object, and when control returns, the native read path accesses state that `detach()` has already released. `C-0080` guided the discovery of Bug 7: a user-supplied profiler timer reenters the profiler by calling `enable()`, freeing state that the profiler's hook dereferences after the timer returns. `C-0077` guided the discovery of Bug 5, a data race on process-global timezone state in Python's `time` module. Finally, Bug 1, a heap overflow in a text codec, is an instance of `C-0036`, a cluster outside the five largest. A long-tail entry yielding a new bug indicates that TAXOSHOP's guidance is not limited to its most populous categories.

Cross-System Bug Discovery. With a $20 budget per interpreter, TAXOSHOP-VA discovers 8 previously unknown bugs in CRuby and 5 in PHP, shown in the lower two segments of Table III. This amounts to an average cost of $2.50 and $4 per bug, respectively.

Pattern Coverage. The 13 bugs found in CRuby and PHP were discovered by three taxonomy entries: `C-0014`, `C-0080`, and `C-0137`. Four bugs match `C-0137`. In CRuby, the boundary is the implicit conversion protocol: Bugs 13–15 call `to_ary` or `to_int` on an argument while a native array routine holds a cached length, and the conversion method shrinks or reallocates the backing buffer before the routine resumes. Bug 21 is the PHP analogue, where `XSLTProcessor` retains a pointer to a stylesheet that is freed while it is still running.

Five bugs belong to `C-0080`. In PHP, the callback is an object lifecycle method invoked from within a library operation: `close()`, `__construct()`, or `closeCursor()` (Bugs 19, 20, 22, 23) reenters an extension that is still executing on the object's native resource, finalizing a statement, context, or archive that the outer operation then uses. Bug 12 is the CRuby instance, in which `Array#flatten` invokes `to_ary` mid-scan.

The remaining four CRuby bugs match `C-0014`. Bugs 11 and 16 arise when `String#unpack` and `encode` mutate the receiver or source string mid-operation; Bugs 17 and 18 when `to_str` and `to_int` resize or shrink a buffer that native code still references.

Absent Patterns. TAXOSHOP-VA found no instances of the concurrency cluster (`C-0077`), or the error-path ownership cluster (`C-0091`) in either interpreter. A plausible explanation is that neither exposes shared mutable state to concurrent execution as CPython does. CRuby serializes threads under a global VM lock, and PHP typically isolates each request. However, absence within a fixed budget does not establish that these patterns do not occur.

Answer to RQ2: TAXOSHOP-VA discovers 23 previously unknown bugs across multiple interpreters with a taxonomy generated only from CPython. These results indicate that the taxonomy is abstract, yet useful enough such that it can meaningfully guide variant analysis.

3) Comparison Study: To measure the contribution of the TAXOSHOP taxonomy, we compare TAXOSHOP-VA against the same variant analyzer under three alternative forms of guidance on CPython: none (Unguided), CWE descriptions (CWE-Guided), and individual CPython bug reports (Instance-Guided). Table IV reports the average number of true-positive (TP) and false-positive (FP) findings per campaign, the time to the first confirmed bug (TTD), and the total campaign time. Findings which use language features to manually trigger sanitizers or campaign-relative duplicates are labeled as false-positives.

TAXOSHOP-VA outperforms all baselines, finding an average of 9.0 bugs per campaign, nearly three times as many as the strongest baseline, CWE-Guided (3.3). Instance-Guided and Unguided find 1.7 and 1.0 bugs, respectively. TAXOSHOP-VA also finds its first bug soonest, after 6.0 minutes on average, compared with 8.7 to 19.3 minutes for the baselines. Since all

TABLE III: Bugs confirmed by development teams. Issue IDs are redacted for anonymized review.

#	System	Issue ID	Class	Description	Fix
1∗	Python	[redacted]	CWE-125	Heap buffer overflow (OOB read of size 1) in _codecs_cn.c hz_decode on trailing ~	+12 -1
2⋆	Python	[redacted]	CWE-787	Out-of-bounds write (SEGV) in mmapmodule.c mmap_ass_subscript via reentrant resize()	+67 -14
3†	Python	[redacted]	CWE-416	Heap use-after-free (WRITE) in bufferedio.c _io__Buffered_read_impl via detach() during read()	
4†	Python	[redacted]	CWE-416	Heap use-after-free in bufferedio.c _bufferedreader_raw_read via detach() during read(n)	+48 -9
5×	Python	[redacted]	CWE-362	Data race on process-global timezone state in timemodule.c get_zone via concurrent tzset()	+99 -4
6†	Python	[redacted]	CWE-476	Member access within null pointer (segfault) in _sqlite/cursor.c _pysqlite_fetch_one_row	+32 -1
7‡	Python	[redacted]	CWE-416	Heap use-after-free (READ) in _lsprof.c ptrace_enter_call via reentrant enable() from external timer	+38 -1
8‡	Python	[redacted]	CWE-416	Heap use-after-free (READ) caused by a reentrant __eq__ freeing a sentinel while being compared	+44 -1
9‡	Python	[redacted]	CWE-416	Heap use-after-free (READ) caused by a reentrant __eq__ freeing a reference while still in use	+34 -1
10‡	Python	[redacted]	CWE-416	Heap use-after-free (READ) from a reentrant function prematurely freeing a sentinel	+44 -1
11†	Ruby	[redacted]	CWE-416	Heap use-after-free (READ) in pack.c pack_unpack_internal via String#unpack mutating the receiver	+20 -0
12‡	Ruby	[redacted]	CWE-125	Heap buffer overflow (READ) in array.c flatten via reentrant to_ary shrinking the array	+137 -2
13⋆	Ruby	[redacted]	CWE-125	Heap buffer overflow (READ) in array.c rb_ary_zip via reentrant to_ary shrinking the receiver	+13 -1
14⋆	Ruby	[redacted]	CWE-125	Heap buffer overflow (READ) + negative-length Array corruption in array.c via to_int	
15⋆	Ruby	[redacted]	CWE-125	Heap buffer overflow (READ) in array.c append_values_at_single via Range endpoint to_int	
16†	Ruby	[redacted]	CWE-416	Heap use-after-free (READ) in transcode.c transcode_restartable0 via encode mutating the source	+28 -3
17†	Ruby	[redacted]	CWE-416	Heap use-after-free (READ) in io_buffer.c io_buffer_get_string via to_str resizing the buffer	
18†	Ruby	[redacted]	CWE-787	Heap out-of-bounds (WRITE) in pack.c pack_pack ('r'/'R') via to_int shrinking the buffer	+16 -1
19‡	PHP	[redacted]	CWE-416	Use-after-free (SEGV) in ext/sqlite3/sqlite3.c SQLite3Stmt::close() finalizing the running VDBE	
20‡	PHP	[redacted]	CWE-416	Heap use-after-free (WRITE) in xpath.c php_xpath_eval via __construct() freeing context	
21⋆	PHP	[redacted]	CWE-416	Heap use-after-free (READ) in xsltprocessor.c php_xsl_apply_stylesheet via freed running stylesheet	+96 -2
22‡	PHP	[redacted]	CWE-416	Use-after-free (SEGV) in php_zip.c php_zipobj_close via close() double-closing the archive	+121 -1
23‡	PHP	[redacted]	CWE-476	Use-of-reset VDBE (SEGV) in pdo_sqlite/sqlite_statement.c via closeCursor() from a UDF callback	

† 0014, ∗ 0036, × 0077, ‡ 0080, ⋆ 0137.

TABLE IV: Comparison of TAXOSHOP-VA with baselines on CPython, averaged over three campaigns. TP and FP are confirmed and false-positive findings per campaign; TTD is the time to the first confirmed bug.

Configuration	TP	FP	TTD (min)	Time (min)
Unguided	1.0	0.0	19.3	53.7
CWE-Guided	3.3	0.0	11.0	61.7
Instance-Guided	1.7	3.3	8.7	60.0
TAXOSHOP-VA	**9.0**	**0.0**	**6.0**	64.0

configurations run for a similar total time (54 to 64 minutes), these differences reflect the effectiveness of the guidance rather than differences in search effort.

Instance-Guided is the only configuration that produces false positives (3.3 per campaign). Given specific bug reports, more than other configurations, the analyzer tended to make reproducers of previously known bugs rather than new discoveries. It also finds fewer bugs than CWE-Guided, even though it receives more specific information. This supports the premise of TAXOSHOP: individual bug instances are too specific to generalize from, and CWE classes are too coarse to locate new bugs, whereas the taxonomy's categories describe mechanisms at a level that is both general and actionable.

Answer to RQ3: Over 3 independent campaigns, TAXOSHOP-VA is able to far outperform other variant analysis techniques.

4) Clustering Strategy: TAXOSHOP clusters mined bug reports in two phases: an embedding pass that groups signatures by cosine similarity, followed by an agentic refinement pass. To evaluate this design, we compare it against each phase in isolation. Figure 4 shows the resulting cluster-size distributions.

All three methods produce long-tailed distributions; however, the clustering characteristics vary substantially. Singletons account for 81.3% of clusters under the embedding-based method, compared with 65.2% under LLM-as-judge and 44.8% under the hybrid method.

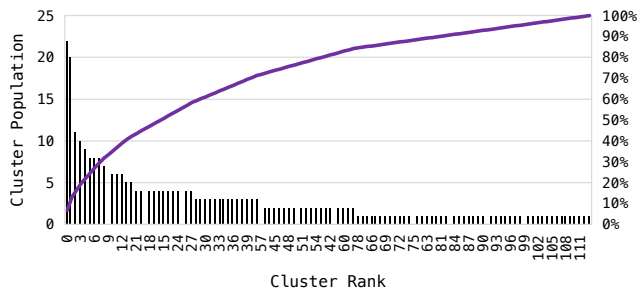
The hybrid method places more reports into populated categories. Its mean cluster size is 3.13, compared with 1.99 for embedding-based clustering and 2.21 for LLM-as-judge. Its largest cluster contains 38 reports, compared with 66 and 38, respectively. Furthermore, 48.2% of reports fall in clusters of five or more members, compared with 40.8% for embedding-based clustering and 38.3% for LLM-as-judge. These results indicate a reduction in fragmentation.

These results support the two-phase design. The embedding pass alone over-fragments the report corpus. The refinement pass corrects both of its failure modes, moving reports out of an over-merged cluster and out of the singleton tail into mechanism-coherent groups. We use the hybrid clustering for the taxonomy evaluated in the remainder of this section.

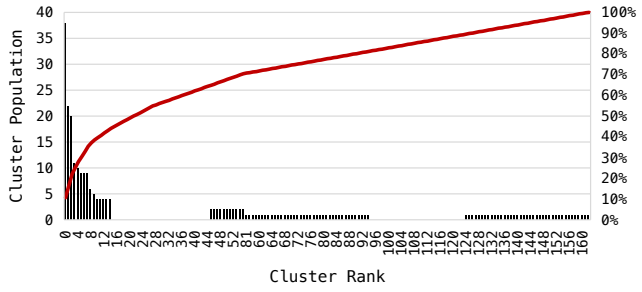
Answer to RQ4: Hybrid clustering produces a more balanced distribution than both LLM-as-judge and embedding methods.

V. CASE STUDIES

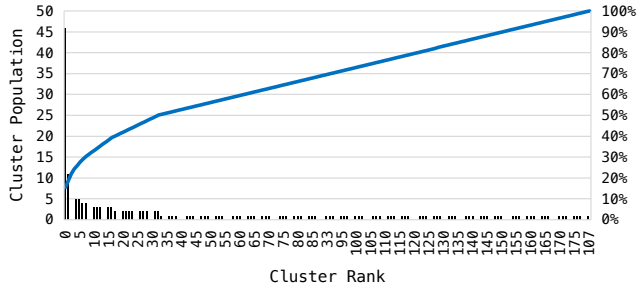
To illustrate how a single taxonomy entry guides discovery across interpreters, we examine three bugs found by TAXOSHOP-VA, all guided by `C-0080` (Figure 5). Figure 6 gives a proof of concept for each. Despite targeting three codebases with unrelated APIs, the three bugs share the same pattern; a native operation retains borrowed state across a call into user code, the user code reenters the interpreter and invalidates that state, and the operation then resumes using it.



(a) Hybrid Clustering



(b) LLM-as-Judge Clustering



(c) Embeddings-based Clustering

Fig. 4: Pareto Distribution of CPython Clusters under three clustering approaches.

A. Case Study 1: CPython

The `_lsprof` profiler allows an external timer callback to run arbitrary Python code on each profiled event. That callback can recursively call `Profiler.enable()`, which is not guarded against reentrancy. The nested activation re-registers the monitoring callbacks and resets the profiler state while the outer event is still relying on its active `ProfilerContext` chain and freelist. When profiling is later disabled, cleanup frees a context that the outer event still references, and the next access is a heap use-after-free.

The bug follows the `C-0080` shape. The outer profiler operation retains borrowed internal state, its native context chain and freelist, across the timer callback. The callback reenters the same subsystem and mutates its lifecycle state. Execution then resumes and cleans up through the invalidated state.

```
# Reentrant invalidation of retained native state during
user-code boundaries.
A native operation retains borrowed objects, raw storage,
cached bounds, iteration state, or resource handles
across user-controlled work. Reentrant mutation
invalidates that state, but execution resumes through the
old view.
## Precondition
The operation captures unowned objects, pointers,
buffers, lengths, indexes, iterators, tables, or resource
handles before conversion, comparison, hashing,
formatting, cleanup, I/O, or callback dispatch that may
execute arbitrary code.
## Critical operation
The operation invokes user-defined behavior or
destruction while continuing to depend on the captured
object lifetime, storage address, structural layout,
bounds, representation, iterator state, or associated
resource.
## Interference
Reentrant code clears, resizes, replaces, detaches,
releases, destroys, or recursively re-enters the owner,
backing storage, resource, or active operation context.
## Invalid assumption
The operation assumes the captured state remains live and
structurally consistent, or that a borrowed object
remains available, after the reentrant boundary.
## Failure
Continuation through stale or null state causes
use-after-free, null dereference, out-of-bounds access,
invalid iteration, corrupted results, or process
termination instead of defined recovery.
## Scope
The common pattern is reentrant invalidation of retained
native state, spanning object lifetimes, buffers, tables,
iterators, resources, exception state, and recursive
context. Some cases use stale or freed state; others use
null or reshaped state. Ownership leaks alone are outside
scope.
```

Fig. 5: Taxonomy entry for `C-0080`, as generated by TAXOSHOP.

B. Case Study 2: CRuby

`Array#flatten` scans the receiver for an element convertible to an array, invoking each candidate's user-defined `to_ary` during the scan. In the PoC, the malicious element follows 2,000 ordinary ones. Its `to_ary` clears the receiver and shrinks the array's heap buffer. However, `flatten` has cached the pre-callback index (`i = 2000`) and resumes by copying that many elements from the now-smaller buffer into a result array sized from the post-clear length. The stale bound makes the copy read past the end of the shrunken buffer and write past the end of the under-allocated result.

This is again `C-0080`, but the retained state is not a pointer. It is the cached length and index that determine the copy size. The `to_ary` callback invalidates that cached bound by shrinking and reallocating the receiver's storage, and the native copy proceeds under the stale bound.

C. Case Study 3: PHP

`ZipArchive::close()` commits an archive through libzip's `zip_close()`, during which libzip invokes the registered progress callback in userland. In the PoC, that callback recursively calls `ZipArchive::close()` on the same object. The wrapper has not yet cleared its native archive pointer, so the nested call runs `zip_close()` on the archive already being closed. The inner close frees the native `zip_t`,

(a) CPython: `_lsprof` (Bug 7)

```
import _lsprof
prof = _lsprof.Profiler()
def _timer():
    try:
        prof.enable()          # reenters the profiler
    except Exception:
        pass
    return 0
prof.__init__(timer=_timer)
prof.enable()
def foo():
    return 1
foo()                          # use-after-free
prof.disable()
```

(b) CRuby: `Array#flatten` (Bug 12)

```
a = (1..2000).to_a
bad = Object.new
$a = a
def bad.to_ary; $a.clear; [1,2,3]; end   # shrinks the
    array
a << bad
a.flatten                        # heap buffer overflow
```

(c) PHP: `ZipArchive::close()` (Bug 22)

```
<?php
$tmp = tempnam(sys_get_temp_dir(), 'zc');
$zip = new ZipArchive();
$zip->open($tmp, ZipArchive::CREATE | ZipArchive::
    OVERWRITE);
for ($i = 0; $i < 64; $i++) {
    $zip->addFromString("f$i.txt", str_repeat('x', 2000));
}
$GLOBALS['zip'] = $zip;
$zip->registerProgressCallback(0.0, function ($r) {
    static $done = false;
    if (!$done) { $done = true; $GLOBALS['zip']->close();
    } // reenters close()
});
$zip->close();                   // use-after-free
```

Fig. 6: PoCs for bugs discovered by TAXOSHOP-VA in each interpreter. All three are instances of `C-0080` (Figure 5): a user-defined callback reenters the interpreter and invalidates state that native code continues to use.

and when the outer `zip_close()` resumes it operates on freed memory and crashes.

The pattern matches `C-0080`. The outer close retains a live native archive handle across a user-controlled callback. The callback reenters the same API and destroys that handle, and the resuming outer close dereferences the invalidated resource.

VI. THREATS TO VALIDITY

Internal Validity. TAXOSHOP uses LLMs, whose outputs are nondeterministic. Different runs may produce different signatures, cluster assignments, and entries. The embeddings phase of the hybrid clustering reduces the nondeterminism slightly.

The LLMs used were likely trained on CPython's public issue tracker, and some taxonomy entries may reflect some of the model's prior knowledge rather than the mined evidence alone. This threat does not apply to the discovered bugs, which were previously unreported. To limit unsupported generalization, TAXOSHOP grounds each entry in the full text of its member reports.

External Validity. We construct a single taxonomy, from 363 memory-safety reports in one project, and evaluate its transfer to two other interpreters. All three are mature interpreters written in C, with similar object models and memory-management strategies. The taxonomy's transfer may not extend to interpreters written in other languages, to other categories of software, or to bug classes other than memory safety. Finally, our results are obtained with specific models (GPT-5.6 Luna and Claude Opus 4.8) and a fixed budget. Other models or budgets may yield different taxonomies and different numbers of discovered bugs.

VII. RELATED WORK

Variant Analysis. Variant analysis finds new bugs that share a root cause with a known one. Early approaches treat a variant as a clone of vulnerable code. SecureSync [28], ReDeBug [11], and VUDDY [7] match syntactic clones at scale. Other systems derive signatures from the vulnerable and patched code [12], [29], [30], from semantically equivalent statements [31], or from taint traces [32], [33]. Such fixed signatures constrain the search to the seed's surface form and struggle when a variant shares only the root cause, as when the same mistake recurs across many builtins of an interpreter [34]. Alternative approaches encode the seed bug as a query over a code representation such as a code property graph [35], [36]. Because writing such queries is expensive, recent work has LLMs infer them [15], [37], [38], [9]. Recent works use LLM agents to hunt variants directly. Big Sleep seeded an agent with a fixed SQLite bug to find a new one [14], and RepoAudit audits repositories by general bug class [39]. GRAPE [4] generalizes a report into a structured pattern, but each pattern is written from a single seed. All of these techniques begin from one bug or a generic bug class. TAXOSHOP instead mines an entire bug history into mechanism-level patterns, targets multiple interpreter implementations, and confirms each candidate with an executable test program rather than the model's judgment.

Interpreter Bug Finding. Fuzzing is the dominant technique for finding bugs in language interpreters. Generation- and mutation-based fuzzers recombine test fragments [40], evolve programs genetically [41], combine grammars with coverage feedback [42], [43], assemble semantically valid inputs from typed fragments [44], [45], and generalize semantics-aware generation across languages [46]. DIE preserves structural aspects of seed tests [47], Fuzzilli mutates an intermediate language to reach JIT compilers [48], and differential oracles catch miscompilations that do not crash [49], [50]. More recent work generates inputs with neural and LLM-based models [51], [52], [53], [54].

VIII. CONCLUSION

We presented TAXOSHOP, an approach that uses LLMs to mine a project's bug history into a taxonomy of recurring bug patterns. TAXOSHOP abstracts away low-level details of

bug reports before clustering the resulting workflow signatures with a hybrid strategy. These clusters are analyzed to produce the final CPython taxonomy. Pairing the taxonomy with a variant analysis pipeline, TAXOSHOP-VA discovers 23 previously unknown bugs across CPython, CRuby, and PHP, evidencing the taxonomy's utility for guided bug discovery.

REFERENCES

[1] W3Techs. Usage statistics and market share of php for websites. https://w3techs.com/technologies/details/pl-php. Accessed: August 20, 2026.

[2] TIOBE Software BV. TIOBE Index for September 2026. https://www.tiobe.com/tiobe-index/, September 2026. Accessed: 2026-09-28.

[3] Yifan Wu, Xiaochuan Yu, and Zhiyun Qian. Melting the flesh of PHP's memory hardening. In *35th USENIX Security Symposium (USENIX Security 26)*, pages 2803–2821, Baltimore, MD, August 2026. USENIX Association.

[4] Han Zheng, Flavio Toffalini, Qiang Liu, and Mathias Payer. Squeezing juicy variant bugs out of modern browsers. In *20th USENIX WOOT Conference on Offensive Technologies (WOOT 26)*, pages 131–151, Baltimore, MD, August 2026. USENIX Association.

[5] Fabian Yamaguchi, Markus Lottmann, and Konrad Rieck. Generalized vulnerability extrapolation using abstract syntax trees. In *Proceedings of the 28th Annual Computer Security Applications Conference*, ACSAC '12, pages 359–368. ACM, 2012.

[6] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, H. Qi, and Jie Hu. VulPecker: An automated vulnerability detection system based on code similarity analysis. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*, ACSAC '16, pages 201–213. ACM, 2016.

[7] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. VUDDY: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy*, pages 595–614. IEEE, 2017.

[8] Shouguo Yang, Zhengzi Xu, Yang Xiao, Zhe Lang, Wei Tang, Yang Liu, Zhiqiang Shi, Hong Li, and Limin Sun. Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis. *ACM Transactions on Software Engineering and Methodology*, 32(6):1–29, 2023.

[9] Claire Wang, Ziyang Li, Saikat Dutta, and Mayur Naik. QLCoder: A query synthesizer for static analysis of security vulnerabilities. In *International Conference on Learning Representations*, 2026.

[10] Arash Ale Ebrahim, Ali Abbasi, and Nils Ole Tippenhauer. DNA: Automated vulnerability variant detection through ai-synthesized queries. In *Proceedings of the 19th European Workshop on Systems Security*, EuroSec '26, pages 74–80. ACM, 2026.

[11] Jiyong Jang, Abeer Agrawal, and David Brumley. ReDeBug: Finding unpatched code clones in entire OS distributions. In *2012 IEEE Symposium on Security and Privacy*, pages 48–62. IEEE, 2012.

[12] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, and Wenchang Shi. MVP: Detecting vulnerabilities using patch-enhanced vulnerability signatures. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 1165–1182. USENIX Association, 2020.

[13] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pages 2329–2344. ACM, 2017.

[14] Big Sleep Team. From Naptime to Big Sleep: Using large language models to catch vulnerabilities in real-world code. Google Project Zero Blog, https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html, 2024. Accessed: 2026-10-10.

[15] Ziyang Li, Saikat Dutta, and Mayur Naik. Iris: Llm-assisted static analysis for detecting security vulnerabilities, 2025.

[16] MITRE. CVE-2025-4516: Use-after-free in CPython's unicode-escape decoder. https://nvd.nist.gov/vuln/detail/CVE-2025-4516, 2025. Accessed: 2026-10-20.

[17] MITRE. CVE-2014-4975: Off-by-one stack buffer overflow in Ruby's `pack.c` `encodes()` function. https://nvd.nist.gov/vuln/detail/CVE-2014-4975, 2014. Accessed: 2026-10-20.

[18] MITRE. CVE-2007-1285: Memory corruption in the Zend engine `zend_hash_init`. https://nvd.nist.gov/vuln/detail/CVE-2007-1285, 2007. Accessed: 2026-10-20.

[19] Python Software Foundation. The Python language reference. https://docs.python.org/3/reference/, 2026. Accessed: 2026-10-10.

[20] Ruby Core Team. Ruby documentation. https://docs.ruby-lang.org/en/master/, 2026. Accessed: 2026-10-10.

[21] The PHP Group. PHP manual. https://www.php.net/manual/en/, 2026. Accessed: 2026-10-10.

[22] Eric Snow. PEP 684 – a per-interpreter GIL. https://peps.python.org/pep-0684/, 2022. Accessed: 2026-10-10.

[23] Sam Gross. PEP 703 – making the global interpreter lock optional in CPython. https://peps.python.org/pep-0703/, 2023. Accessed: 2026-10-10.

[24] Thomas Wouters, Matt Page, and Sam Gross. PEP 779 – criteria for supported status for free-threaded Python. https://peps.python.org/pep-0779/, 2025. Accessed: 2026-10-10.

[25] GitHub. CodeQL. https://codeql.github.com/, 2026. Accessed: 2026-10-10.

[26] Semgrep, Inc. Semgrep. https://semgrep.dev/, 2026. Accessed: 2026-10-10.

[27] Junyoung Park and Insu Yun. Agentic fuzzing: Opportunities and challenges, 2026.

[28] Nam H. Pham, Tung Thanh Nguyen, Hoan Anh Nguyen, and Tien N. Nguyen. Detection of recurring software vulnerabilities. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2010.

[29] Seunghoon Woo, Hyunji Hong, Eunjin Choi, and Heejo Lee. MOVERY: A precise approach for modified vulnerable code clone discovery from modified open-source software components. In *31st USENIX Security Symposium*, pages 3037–3053, 2022.

[30] Seunghoon Woo, Eunjin Choi, Heejo Lee, and Hakjoo Oh. V1SCAN: Discovering 1-day vulnerabilities in reused C/C++ open-source software components using code classification techniques. In *32nd USENIX Security Symposium*, pages 6541–6556, 2023.

[31] Kaifeng Huang, Chenhao Lu, Yiheng Cao, Bihuan Chen, and Xin Peng. VMUD: Detecting recurring vulnerabilities with multiple fixing functions via function selection and semantic equivalent statement matching. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 3958–3972, 2024.

[32] Wooseok Kang, Byoungho Son, and Kihong Heo. TRACER: Signature-based static analysis for detecting recurring vulnerabilities. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.

[33] Siyue Feng, Yueming Wu, Wenjie Xue, Sikui Pan, Deqing Zou, Yang Liu, and Hai Jin. FIRE: Combining multi-stage filtering with taint analysis for scalable recurring vulnerability detection. In *33rd USENIX Security Symposium*, pages 1867–1884, 2024.

[34] Yiheng Cao, Susheng Wu, Ruisi Wang, Bihuan Chen, Yiheng Huang, Chenhao Lu, Zhuotong Zhou, and Xin Peng. Recurring vulnerability detection: How far are we? *Proceedings of the ACM on Software Engineering*, 2(ISSTA), 2025.

[35] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy (S&P)*, 2014.

[36] Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. QL: Object-oriented queries on relational data. In *30th European Conference on Object-Oriented Programming (ECOOP)*, 2016.

[37] Ahmed Lekssays, Hamza Mouhcine, Khang Tran, Ting Yu, and Issa Khalil. LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models. In *34th USENIX Security Symposium*, pages 489–507, 2025.

[38] Chenyuan Yang, Zijie Zhao, Zichen Xie, Haoyu Li, and Lingming Zhang. KNighter: Transforming static analysis with LLM-synthesized checkers. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, 2025.

[39] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. RepoAudit: An autonomous LLM-agent for repository-level code auditing. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 21083–21100, 2025.

[40] Christian Holler, Kim Herzig, and Andreas Zeller. Fuzzing with code fragments. In *21st USENIX Security Symposium*, 2012.

[41] Spandan Veggalam, Sanjay Rawat, Istvan Haller, and Herbert Bos. IFuzzer: An evolutionary interpreter fuzzer using genetic programming. In *European Symposium on Research in Computer Security (ESORICS)*, 2016.

[42] Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. NAUTILUS: Fishing for deep bugs with grammars. In *Network and Distributed System Security Symposium (NDSS)*, 2019.

[43] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. Superion: Grammar-aware greybox fuzzing. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019.

[44] HyungSeok Han, DongHyeon Oh, and Sang Kil Cha. CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines. In *Network and Distributed System Security Symposium (NDSS)*, 2019.

[45] Christopher Salls, Chani Jindal, Jake Corina, Christopher Kruegel, and Giovanni Vigna. Token-level fuzzing. In *30th USENIX Security Symposium*, 2021.

[46] Yongheng Chen, Rui Zhong, Hong Hu, Hangfan Zhang, Yupeng Yang, Dinghao Wu, and Wenke Lee. One engine to fuzz 'em all: Generic language processor testing with semantic validation. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.

[47] Soyeon Park, Wen Xu, Insu Yun, Daehee Jang, and Taesoo Kim. Fuzzing JavaScript engines with aspect-preserving mutation. In *IEEE Symposium on Security and Privacy (S&P)*, 2020.

[48] Samuel Groß, Simon Koch, Lukas Bernhard, Thorsten Holz, and Martin Johns. FUZZILLI: Fuzzing for JavaScript JIT compiler vulnerabilities. In *Network and Distributed System Security Symposium (NDSS)*, 2023.

[49] Lukas Bernhard, Tobias Scharnowski, Moritz Schloegel, Tim Blazytko, and Thorsten Holz. JIT-Picking: Differential fuzzing of JavaScript engines. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.

[50] Junjie Wang, Zhiyi Zhang, Shuang Liu, Xiaoning Du, and Junjie Chen. FuzzJIT: Oracle-enhanced fuzzing for JavaScript engine JIT compiler. In *32nd USENIX Security Symposium*, 2023.

[51] Suyoung Lee, HyungSeok Han, Sang Kil Cha, and Sooel Son. Montage: A neural network language model-guided JavaScript engine fuzzer. In *29th USENIX Security Symposium*, 2020.

[52] Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Xiaoyang Sun, Lizhong Bian, Haibo Wang, and Zheng Wang. Automated conformance testing for JavaScript engines via deep compiler fuzzing. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021.

[53] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal fuzzing with large language models. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

[54] Jueon Eom, Seyeon Jeong, and Taekyoung Kwon. Fuzzing JavaScript interpreters with coverage-guided reinforcement learning for LLM-based mutation. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024.